



Javier Urbaneja Benítez

03/10/2026

STAGE 1

① Missing "accept()" argument:

The server didn't compile because "accept()" was called with only two arguments. GCC reported "too few arguments to function 'accept'". I added "8 cli-len" as the third argument and now it compiles. I measured the fix by compiling with "-Wall" and seeing that there's not any error.

② wrong address family:

The server failed at startup with "bind: Address family not supported by protocol" because "serve_addr.sin_family" was set to AF_UNIX, even though the socket uses AF_INET. I changed it to AF_INET and I measured the fix by restarting the server and checking that "bind()" doesn't fail.



③ client disconnect and SIGPIPE

A forked child could terminate if the client disconnect while the server was preparing its response. I changed the server to ignore SIGPIPE so a failed "write()" wouldn't ~~the~~ terminate the child. I measure the fix by disconnecting to a client while server was replying and checking that now the child terminate correctly.

• STAGE 2

- The time stays at about 6 seconds even though the number of (child) client increases because each client is handled by a different child process, so we don't need to wait until the rest of clients had finished their connections.

• STAGE 3

- ① The normal client received the (greeting) message correctly, the "-- leave" disconnected immediately. ~~But the~~ The server reply to both because we ignored the SIGPIPE.
- ② After three client finished, "pgrep -c -x PZ -conc -server" shows 4 processes, which is correct because we have the server plus the three children in z state.
- ③ After killing one child while it is waiting its response, the server doesn't print "child... finished" but it still accepting new clients because we only kill the child, not the parent.